

Numpy

Aquest notebook és una versió adaptada d'Abhay Parashar
(<https://www.kaggle.com/code/abhayparashar31/best-numpy-functions-for-data-science-50>).

===== TAULA DE CONTINGUTS =====

- Mètodes de Creació d'Arrays
- Accés a Arrays
- Operacions amb Arrays
- Substitució de Valors dins d'un Array
- Filtratge

Mètodes de Creació d'Arrays

Array

Crea arrays d'una dimensió o multidimensionals.

```
array(object, dtype=None, *, copy=True, order='K', subok=False, ndmin=0)
```

```
In [1]: import numpy as np
```

```
In [2]: # Crear un array  
a = np.array([1, 2, 3, 4, 5, 6])  
  
print(a)  
print(type(a))
```

```
[1 2 3 4 5 6]  
<class 'numpy.ndarray'>
```

Arange

Retorna valors enters equiespaiats en un interval donat amb un cert pas (1 per defecte).

```
np.arange(start, stop, step, dtype=None)
```

- **Paràmetres importants**

`step`: Espai entre valors.

```
In [3]: np.arange(5)
```

```
Out[3]: array([0, 1, 2, 3, 4])
```

```
In [4]: np.arange(5, 10)
```

```
Out[4]: array([5, 6, 7, 8, 9])
```

```
In [5]: np.arange(5, 10, 2)
```

```
Out[5]: array([5, 7, 9])
```

Random.randint

Genera n mostres aleatòries enteres dins d'un rang.

```
np.random.randint(low, high=None, size=None, dtype=int)
```

```
In [6]: np.random.randint(10)
```

```
Out[6]: 8
```

```
In [7]: np.random.randint(5, 10, 12)
```

```
Out[7]: array([8, 5, 8, 5, 9, 8, 9, 9, 8, 7, 9, 7], dtype=int32)
```

Random.random

Genera n mostres aleatòries decimals entre 0 i 1.

```
np.random(size=None)
```

```
In [8]: np.random.random(3)
```

```
Out[8]: array([0.63488218, 0.4043823 , 0.89352135])
```

Array de zeros

Crea un array de 0.

```
np.zeros(shape, dtype=float, order='C')
```

- ▀ **Paràmetres importants**

`shape` : forma de l'array resultant.

`dtype` : tipus de dades desitjat de l'array resultant. 'int' o per defecte 'float'

```
In [9]: np.zeros(5)
```

```
Out[9]: array([0., 0., 0., 0., 0.])
```

```
In [10]: np.zeros((2, 3), dtype='int')
```

```
Out[10]: array([[0, 0, 0],
                [0, 0, 0]])
```

Array d'Uns

Crea un array d'1.

```
np.ones(shape, dtype=None, order='C')
```

```
In [11]: np.ones((3,4))
```

```
Out[11]: array([[1., 1., 1., 1.],  
                [1., 1., 1., 1.],  
                [1., 1., 1., 1.]])
```


Array amb un nombre escollit

Crea un array n-dimensional amb un valor.

```
np.full(shape, fill_value, dtype=None)
```

- **Paràmetres importants**

`fill_value` : Valor amb el qual omplir l'array.

```
In [12]: np.full((2, 4), fill_value=3)
```

```
Out[12]: array([[3, 3, 3, 3],  
                [3, 3, 3, 3]])
```

Forma (Shape)

```
In [13]: a = np.random.randint(1, 10, 5)  
a
```

```
Out[13]: array([3, 4, 4, 5, 6], dtype=int32)
```

```
In [14]: a.shape
```

```
Out[14]: (5,)
```

```
In [15]: a = np.random.randint(1, 10, (2, 3))  
a
```

```
Out[15]: array([[3, 1, 3],  
               [3, 7, 9]], dtype=int32)
```

```
In [16]: a.shape
```

```
Out[16]: (2, 3)
```

Transposada

In [17]:

```
a
```

Out[17]:

```
array([[3, 1, 3],  
       [3, 7, 9]], dtype=int32)
```

In [18]:

```
a.T
```

Out[18]:

```
array([[3, 3],  
       [1, 7],  
       [3, 9]], dtype=int32)
```

Accés a Arrays

```
In [19]: a = np.array([1, 2, 3, 4, 5, 6])
```

```
In [20]: # Indexació  
a[2]
```

```
Out[20]: np.int64(3)
```

```
In [21]: # Indexació negativa  
a[-1]
```

```
Out[21]: np.int64(6)
```

```
In [22]: a = np.array([1, 2, 3, 4, 5, 6])
```

```
In [23]: # Slicing  
a[1:4]
```

```
Out[23]: array([2, 3, 4])
```

```
In [24]: # Slicing des de 0  
a[:4]
```

```
Out[24]: array([1, 2, 3, 4])
```

```
In [25]: # Slicing fins al final  
a[2:]
```

```
Out[25]: array([3, 4, 5, 6])
```

```
In [26]: a = np.array([1, 2, 3, 4, 5, 6])
```

```
In [27]: # Slicing amb pas  
a[1:4:2]
```

```
Out[27]: array([2, 4])
```

```
In [28]: # Slicing amb pas  
a[:, :2]
```

```
Out[28]: array([1, 3, 5])
```

```
In [29]: a = np.random.randint(1, 10, size=(2, 4))  
a
```

```
Out[29]: array([[8, 6, 5, 7],  
               [9, 9, 1, 2]], dtype=int32)
```

```
In [30]: # Accedir al primer índex  
a[0]
```

```
Out[30]: array([8, 6, 5, 7], dtype=int32)
```

```
In [31]: # Accedir a l'element i-èsim  
a[0][2]
```

```
Out[31]: np.int32(5)
```

```
In [32]: # Accedir a l'element i-èsim  
# Igual que abans però més eficient!  
a[0, 2]
```

```
Out[32]: np.int32(5)
```

```
In [33]: # Tota la columna amb índex 2  
a[:, 2]
```

```
Out[33]: array([5, 1], dtype=int32)
```



```
In [34]: a = np.array([[4, 1, 9, 7],  
                      [6, 6, 7, 5]])
```

```
In [35]: # Assignació de valor  
print(a)  
print()  
  
a[0, 0] = 2  
  
print(a)
```

```
[[4 1 9 7]  
 [6 6 7 5]]
```

```
[[2 1 9 7]  
 [6 6 7 5]]
```

```
In [36]: # Assignació amb slicing  
a[0, 1:] = 8  
a
```

```
Out[36]: array([[2, 8, 8, 8],  
               [6, 6, 7, 5]])
```

Operacions amb Arrays

Operacions matemàtiques

```
In [37]: a = np.array([1, 2, 3, 4, 5, 6])
```

```
In [38]: a + 3
```

```
Out[38]: array([4, 5, 6, 7, 8, 9])
```

```
In [39]: a - 1
```

```
Out[39]: array([0, 1, 2, 3, 4, 5])
```

```
In [40]: a * 2
```

```
Out[40]: array([ 2,  4,  6,  8, 10, 12])
```

```
In [41]: a / 4
```

```
Out[41]: array([0.25, 0.5 , 0.75, 1.  , 1.25, 1.5  ])
```

```
In [42]: a % 2
```

```
Out[42]: array([1, 0, 1, 0, 1, 0])
```

Min

Retorna el valor mínim de l'array.

```
np.min(a,axis=None,out=None,keepdims=<no value>,initial=<no value>,where=<no value>)
```

- **Paràmetres importants**

axis: eix sobre el qual operar.

```
In [43]: a = np.array([3, 1, 5, 3, 4, 5, 6, 2])  
np.min(a)
```

```
Out[43]: np.int64(1)
```

```
In [44]: a = np.array([[3, 1, 5, 3], [4, 5, 6, 2]])

print(a)
print()
print("Mínim global:", np.min(a))
print("Mínim per columnes:", np.min(a, axis=0))
print("Mínim per files:", np.min(a, axis=1))
```

```
[[3 1 5 3]
 [4 5 6 2]]
```

```
Mínim global: 1
Mínim per columnes: [3 1 5 2]
Mínim per files: [1 2]
```

- Molts mètodes de numpy es poden cridar com a `np.min(arr)` o `arr.min()`.
- Això no és només per a `min` sinó per a molts altres mètodes.

In [45]:

```
a = np.array([1, 2, 3])
```

```
print(np.min(a))
```

```
print(a.min())
```

```
1
```

```
1
```

Max

Retorna el valor màxim de l'array.

```
np.max(a,axis=None,out=None)
```

```
In [46]: a = np.array([[3, 1, 5, 3], [4, 5, 6, 2]])

print(a)
print()
print("Màxim global:", np.max(a))
print("Màxim per columnes:", np.max(a, axis=0))
print("Màxim per files:", np.max(a, axis=1))
```

```
[[3 1 5 3]
 [4 5 6 2]]
```

```
Màxim global: 6
Màxim per columnes: [4 5 6 3]
Màxim per files: [5 6]
```

Sum

Retorna la suma dels elements de l'array.

```
In [47]: a = np.array([[3, 1, 5, 3], [4, 5, 6, 2]])

print(a)
print()
print("Suma global:", np.sum(a))
print("Suma per columnes:", np.sum(a, axis=0))
print("Suma per files:", np.sum(a, axis=1))
```

```
[[3 1 5 3]
 [4 5 6 2]]
```

```
Suma global: 29
Suma per columnes: [ 7  6 11  5]
Suma per files: [12 17]
```


Unique

Retorna un array amb tots els elements únics ordenats.

```
np.unique(ar, return_index=False, return_inverse=False, return_counts=False, axis=None)
```

In [48]:

```
a
```

Out[48]:

```
array([[3, 1, 5, 3],  
       [4, 5, 6, 2]])
```

In [49]:

```
np.unique(a)
```

Out[49]:

```
array([1, 2, 3, 4, 5, 6])
```

In [50]:

```
np.unique(a, return_counts=True)
```

Out[50]:

```
(array([1, 2, 3, 4, 5, 6]), array([1, 1, 2, 1, 2, 1]))
```

Reshape

És una de les funcions més utilitzades de NumPy. Retorna un array amb les mateixes dades però amb una nova forma.

```
np.reshape(shape)
```

```
In [51]: A = np.random.randint(15, size=(4, 3))  
A
```

```
Out[51]: array([[14,  5,  2],  
                [ 9, 14,  5],  
                [ 3,  7, 11],  
                [12, 14,  1]], dtype=int32)
```

```
In [52]: A.reshape(3, 4)
```

```
Out[52]: array([[14,  5,  2,  9],
                [14,  5,  3,  7],
                [11, 12, 14,  1]], dtype=int32)
```

```
In [53]: # ApLanar
A.reshape(-1)
```

```
Out[53]: array([14,  5,  2,  9, 14,  5,  3,  7, 11, 12, 14,  1], dtype=int32)
```

argmax & argmin

Retorna l'índex de l'element màxim de l'array.

Es pot utilitzar per obtenir l'índex de les etiquetes predites amb alta probabilitat en problemes de classificació.

```
np.argmax(a, axis=None, out=None)
```

```
In [55]: a = np.array([0.12, 0.19, 0.64, 0.05])  
np.argmax(a)
```

```
Out[55]: np.int64(2)
```

Retorna l'índex de l'element més baix de l'array.

```
np.argmin(a, axis=None, out=None)
```

```
In [56]: a = np.array([0.12, 0.19, 0.64, 0.05])  
np.argmin(a)
```

```
Out[56]: np.int64(3)
```

Sort

Retorna un array ordenat.

```
np.sort(a, axis=-1, kind=None, order=None)
```

```
In [57]: a = np.array([2, 3, 1, 7, 4, 5])  
np.sort(a)
```

```
Out[57]: array([1, 2, 3, 4, 5, 7])
```

```
In [58]: # no ha canviat  
a
```

```
Out[58]: array([2, 3, 1, 7, 4, 5])
```

```
In [59]: a.sort()
```

```
In [60]: a
```

```
Out[60]: array([1, 2, 3, 4, 5, 7])
```

Abs

Retorna els valors absoluts dels elements d'un array.

```
In [61]: A = np.array([[1, -3, 4], [-2, -4, 3]])  
np.abs(A)
```

```
Out[61]: array([[1, 3, 4],  
               [2, 4, 3]])
```


Round

Arrodoneix els valors decimals a un nombre especificat de decimals.

```
np.round(a, decimals=0, out=None)
```

- **Paràmetres importants**

`decimals` : Nombre de decimals a mantenir.

```
In [62]: a = np.random.random(size=(3,4))  
a
```

```
Out[62]: array([[0.2422464 , 0.13919913, 0.61467139, 0.38027184],  
               [0.00564905, 0.71865252, 0.44063644, 0.43713275],  
               [0.43991643, 0.3840315 , 0.09509601, 0.70220006]])
```

```
In [63]: np.round(a, decimals=0)
```

```
Out[63]: array([[0., 0., 1., 0.],  
               [0., 1., 0., 0.],  
               [0., 0., 0., 1.]])
```

```
In [64]: np.round(a, decimals=1)
```

```
Out[64]: array([[0.2, 0.1, 0.6, 0.4],  
               [0. , 0.7, 0.4, 0.4],  
               [0.4, 0.4, 0.1, 0.7]])
```

```
In [65]: np.round(a, decimals=2)
```

```
Out[65]: array([[0.24, 0.14, 0.61, 0.38],  
               [0.01, 0.72, 0.44, 0.44],  
               [0.44, 0.38, 0.1 , 0.7 ]])
```

Any

Comprova si almenys un dels valors booleans d'un array és `True`.

```
In [66]: a = np.array([False, True, False])  
  
np.any(a)
```

```
Out[66]: np.True_
```

All

Comprova si tots els valors booleans d'un array són `True`.

```
In [67]: a = np.array([False, True, False])  
  
np.all(a)
```

```
Out[67]: np.False_
```

```
In [68]: b = np.array([True, True, True])  
  
np.all(b)
```

```
Out[68]: np.True_
```

Filtratge

```
In [69]: a = np.array([3, 5, 4, 2])
```

```
In [70]: # Exemple amb filtratge booleà  
bool_list = [True, False, True, False]  
a[bool_list]
```

```
Out[70]: array([3, 4])
```

```
In [71]: a > 3
```

```
Out[71]: array([False,  True,  True, False])
```

```
In [72]: a[a > 3]
```

```
Out[72]: array([5, 4])
```

```
In [73]: # Condició AND  
a[(a > 2) & (a < 5)]
```

```
Out[73]: array([3, 4])
```

```
In [74]: # Condició OR  
a[(a > 2) | (a < 5)]
```

```
Out[74]: array([3, 5, 4, 2])
```

```
In [75]: # Substituir amb un filtre  
a = np.array([3, 5, 4, 2])  
a[a > 3] = 0  
a
```

```
Out[75]: array([3, 0, 0, 2])
```


Where

Retorna els índexs d'un array on es compleix una condició.

```
np.where(condition, [x, y])
```

-  **Paràmetres importants** `condition`: condició a complir. El `True` retorna x , en cas contrari y .

```
In [76]: a = np.array([3, 5, 3, 2])  
         np.where(a == 3)
```

```
Out[76]: (array([0, 2]),)
```

```
In [77]: a = np.arange(12).reshape(4,3)
a
```

```
Out[77]: array([[ 0,  1,  2],
               [ 3,  4,  5],
               [ 6,  7,  8],
               [ 9, 10, 11]])
```

```
In [78]: np.where(a > 5)
```

```
Out[78]: (array([2, 2, 2, 3, 3, 3]), array([0, 1, 2, 0, 1, 2]))
```

```
'''
ELEMENT [2][0] (2NA FILA COLUMNNA 0) 6 > 5, RETORNA
ELEMENT [2][1] (2NA FILA COLUMNNA 1) 7 > 5, RETORNA
.
ELEMENT [3][2] (3A FILA COLUMNNA 2) 11 > 5, RETORNA
'''
```

```
In [79]: a[np.where(a > 5)]
```

```
Out[79]: array([ 6,  7,  8,  9, 10, 11])
```